F21SC: Industrial Programming CW1: Web Browser

Ann Erinjeri – H00418805

INTRODUCTION

This report aims to describe the architecture, implementation, and operation of a C# program for a simple web browser that uses Windows Forms for the graphical user interface (GUI) and the.NET Framework (version 4.8).

My remit is to replicate the main features of a modern multi-user-friendly web browser, including gathering and presenting HTML source code, sending, receiving and displaying HTTP request messages, controlling persistent configurations like the homepage, browsing history, and bookmarks and navigating to online pages using URLs, all backed by a database.

Assumptions

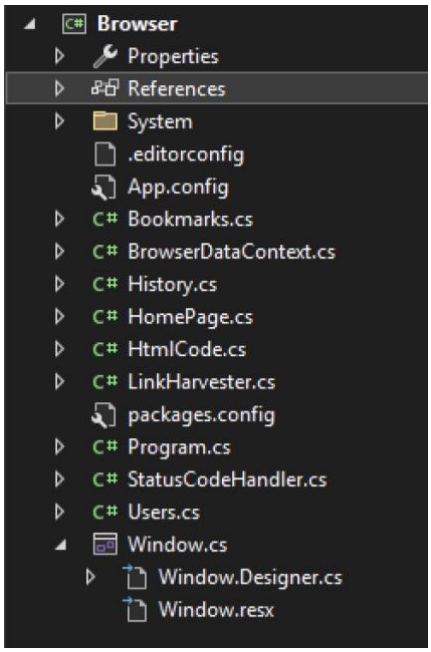
1. The user is presumed to be running the program on a Windows computer with the proper.NET Framework 4.8 runtime installed and compatible hardware (CPU, memory).
2. In order for the system to successfully process HTTP requests and retrieve online content, a steady internet connection is assumed.
3. The websites are presumed to be openly accessible, meaning that content can be seen without the need for cookies, JavaScript, authentication, or sophisticated client-side rendering.
4. The working directory of the application, which contains the SQLite database file, is presumed to be writable, enabling the browser to store and modify persistent data like bookmarks, history, and homepage preferences.
5. It is expected that users would enter well-formed, legitimate URLs (such as "https://"), and that invalid inputs will be handled by error alerts and validation.
6. The local SQLite file (browser.db) is not thought to be altered, corrupted, or removed between sessions, assuming that database integrity stays constant.
7. Although numerous profiles can be generated, it is believed that the program will operate in a single-user environment, with only one user active at a time.
8. It is presumed that the user is free from firewall or proxy constraints that prevent outgoing HTTP requests and has sufficient rights to run the software and access network resources.
9. The browser is not optimized for concurrent queries or heavy data streaming; it is expected that system responsiveness and performance are adequate to manage small to medium-sized webpages.
10. Error dialogs and status messages are thought to give users enough information about problems like timeouts, invalid URLs, or missing pages.
11. Third-party libraries like Microsoft.VisualBasic, System.Data.SQLite, and System.Data.SQLite.Linq are presumed to be correctly installed and compatible with the target runtime environment.
12. The database and application files are presumed to stay in the same directory structure, guaranteeing that all relative routes used to access the database are valid throughout sessions.

REQUIREMENTS CHECKLIST

FEATURE	STATUS
Sending HTTP requests and displaying the contents of the response	Fully implemented
Displaying the HTTP response status code	Fully implemented
Displaying the web page's title	Fully implemented
Reload the current web page	Fully implemented
Setting the home page	Fully implemented
Adding pages to bookmarks list	Fully implemented
Naming bookmarks	Fully implemented
Deleting bookmarks	Fully implemented
Removing bookmarks	Fully implemented
Navigating to bookmarks	Fully implemented
Maintaining history	Fully implemented
Navigating to previous/next page in history	Partially implemented – Backtracking only allowed to websites visited within the session.
Navigating to a selected page in history	Fully implemented
List five urls from the webpage in the side panel	Fully implemented
Settings (home page, bookmarks, history) persist across sessions	Fully implemented

DESIGN CONSIDERATIONS

Class Organization (Elaborated In Developer Guide)

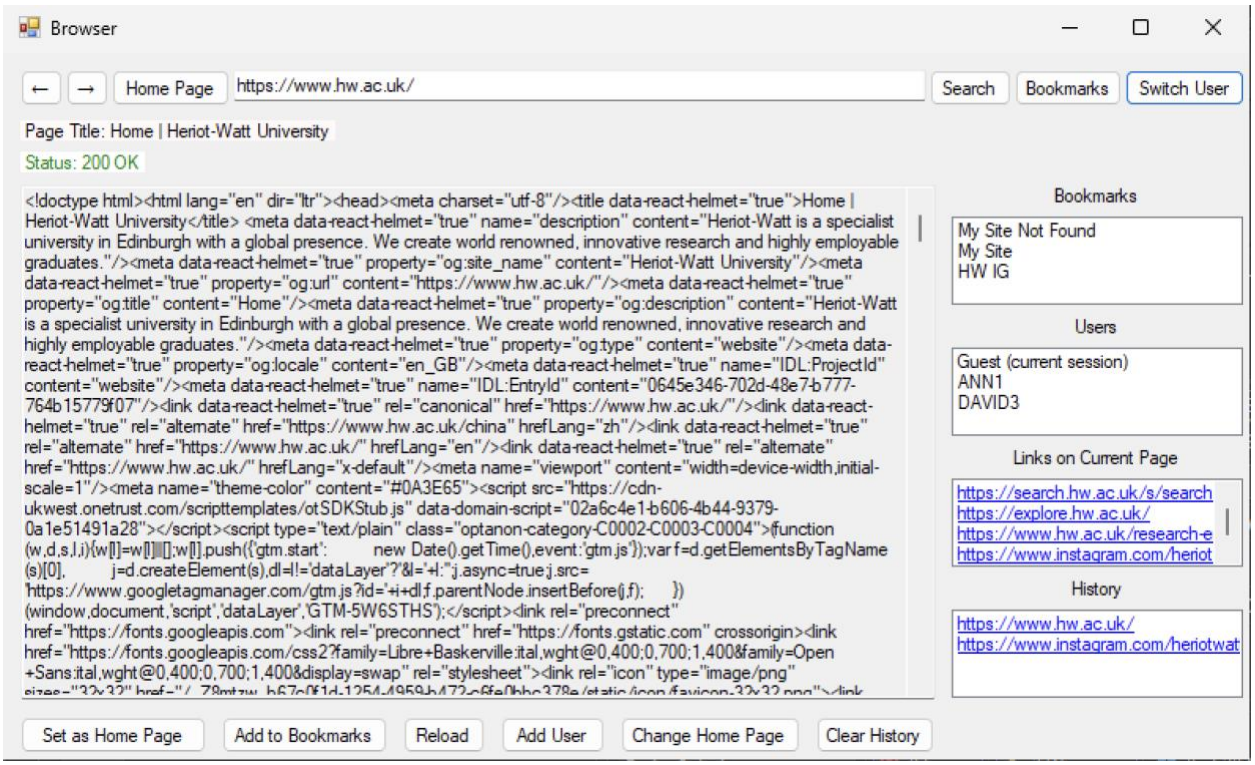


Choice of Data Structures

The browser's lightweight design was coupled with straightforward data structures that support its functionality. To facilitate iteration and quick access, lists (List<string>) were utilized to hold sequential data, including browser history and extracted hyperlinks. For the Back and Forward navigation system, stacks (Stack<string>) were used to ensure that the order was reversed while navigating between pages. Isolated browsing sessions for per profile were made possible by the introduction of dictionaries (Dictionary<string, Stack<string>>) to manage user-specific navigation stacks. In order to provide organized access to the SQLite database, persistent storage activities were also handled by ADO.NET classes like SQLiteCommand and SQLiteConnection. The simplicity, speed, and applicability of these data structures for effectively managing small-scale, real-time browser operations led to their selection.

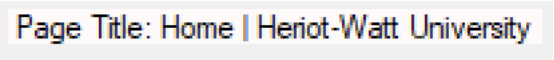
GUI Design

1. Overall View



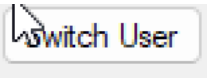
2. GUI vs. Requirements

BACK/FORWARD NAVIGATION



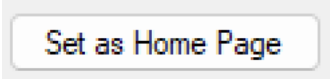
PAGE TITLE DISPLAY

COLOR CODED STATUS CODE – ADDITIONAL MODIFICATION



SWITCH USER

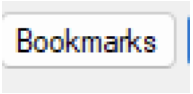
HOME PAGE LOADING



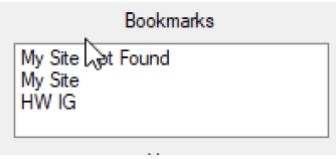
HOME PAGE SETUP

HOME PAGE EDITING

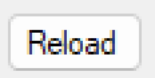
BOOKMARKS SET UP



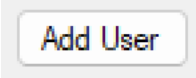
BOOKMARKS DISPLAY TOGGLE



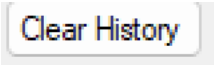
BOOKMARKS DISPLAY



RELOAD



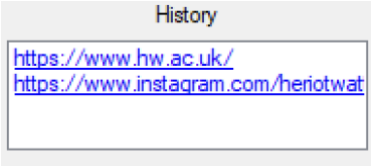
USER CREATION



CLEAR HISTORY – ADDITIONAL FEATURE

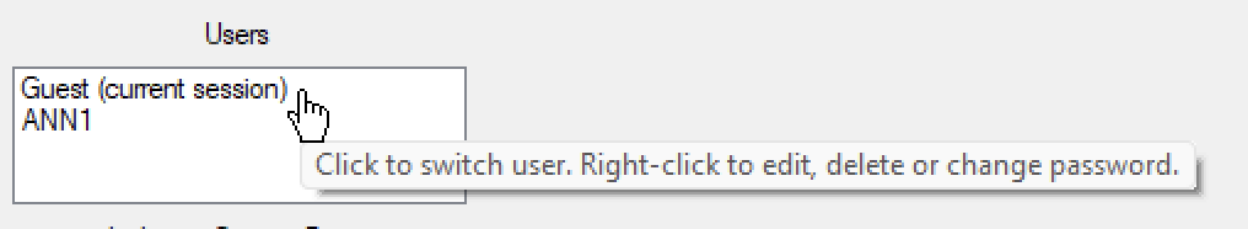
USERS DISPLAY

LINKS FROM WEBPAGE DISPLAY



HISTORY DISPLAY

TOOLTIPS – ADDITIONAL FUNCTIONALITY



HTML CODE DISPLAY

<u>Advanced Language Constructs and Techniques</u>

1. Asynchronous HTTP Requests: in **StatusCodeHandler -** Improve Responsiveness
2. Regular Expressions: in **LinkHarvester** - Extract Hyperlinks
3. Exception Handling: All database and network operations have try-catch blocks – Increase User-friendliness
4. Parameterized Queries: in all SQLite commands - Ensure query reliability.
5. Encapsulation: Private fields and public - Protect Data Integrity.

Performance and Reliability Considerations

1. Error feedback has been done rigorously to ensure that the application does not crash due to invalid inputs or other warnings.

Advanced Functionalities

1. Multi-user functionality has been fully implemented with the SQLite database. Each user has their own unique username and password, which they require to switch to, edit or delete the profile. The passwords are hashed and stored in the database (browser.db). Each user has their own browsing history, bookmarks and homepage. The user is also given an additional feature to change their password.
2. Because of its ease of use, zero-configuration setup, and compatibility with.NET via the System.Data.SQLite library, SQLite was selected for our database.
   Limitation: Problems with LINQ Integration
   LINQ to SQLite (System.Data.SQLite.Linq) was originally supposed to be used to create the database layer. This would have allowed for object-relational mapping (ORM) capabilities and strongly-typed queries. Unfortunately, due to persistent connection issues, this method was dropped.
   The BrowserDataContext class was used to programmatically initialize the database within the project, ensuring that all necessary tables are initialized immediately upon application startup. This ensures uniformity across all environments and eliminates the need for manual database setup.

DEVELOPER GUIDE

Class Design

1. The Program.cs file is the main entry point of the browser. It initializes and launches the GUI by creating and running an instance of the main form, BrowserWindow.

2. The Window.cs file contains the main logic and controller class for the entire browser.
   It defines how the GUI components such as buttons, text boxes, list boxes, and labels behave upon user interaction. It connects the front-end GUI with the backend logic of the program.

   Some key features implemented through Window.cs:

   - Search Function
   - Reload Function
   - Back/Forward navigation
   - Integration of bookmarks, history, link harvester and homepage.
   - User Interaction Handling
   - Context Menus
   - Tooltip integration

3. StatusCodeHandler class checks whether a website is reachable, sends HTTP requests using the HttpClient class from the System.Net.Http namespace, gets and reports the HTTP status code with the reason phrase.

4. The HtmlCode class makes HTTP requests, reads the server's response and returns the raw HTML code, which is displayed in the browser. The status code, title and links on the page are also obtained from the raw HTML code.

5. The HomePage class handles homepage storage and retrieval through the SQLite database (browser.db).

6. The History class maintains the user's browser history, stores it in the SQLite database (browser.db) and displays it. It ensures that all visited pages are appended to the history display, avoiding consecutive duplicates, and facilitating the option to clear browsing history.
7. The Bookmarks class manages functionalities like adding, editing, deleting, and navigating to the user's bookmarks. Each user has their unique set of bookmarks which are stored in the SQLite database (browser.db), linked with their UserId.

8. LinkHarvester class harvests upto 5 hyperlinks from the HTML code of a webpage. It reads through the raw HTML code, matches all anchor tags with href attributes, and collects valid links that start with https://.

9. Advanced Feature: The Users class enables multi-user functionality. It handles user-related functionalities like user creation, deletion, editing username, password creation, password hashing, password verification, password editing and loads bookmarks, history and homepage of active user, stored in the SQLite database. Passwords are hashed and stored using SHA-256 hashing.

10. Advanced Feature: The BrowserDataContext class implements database storage. This class manages the database functionality. It handles creation, initialization, and maintainence of the connection to the SQLite database (browser.db), which ensures that all user-related data such as bookmarks, history, homepages, and user credentials persists across sessions.

11. GUI: The Window.Designer.cs file is an auto-generated file that defines and initializes all GUI components of the browser. GUI elements were modified using the form designer.

<u>Library Overview</u>

The libraries used to execute the HTTP communication:

1. **System.Net.Http** is a built-in .NET library used to send asynchronous HTTP GET requests (GetAsync()).
   Occurrence: **StatusCodeHandler** class

2. **System.Net** is a built-in .NET library used to obtain the raw HTML source code of the requested URL.
   Occurrence: **HtmlCode** class

Error Handling is facilitated through WebException and HttpRequestException.

Libraries Used to Make the Browser Settings Persistent (Homepage, History, and Bookmarks):

1. **System.Data.SQLite** is used to store and retrieve persistent browser data relevant to active user such as homepages, browsing history, bookmarks, and user information in the SQLite Database (browser.db).

   Occurrence: **BrowserDataContext, Bookmarks, History, HomePage,** and **Users** classes

2. **SQLiteCommand** and **SQLiteConnection (ADO.NET Components)** are used to perform Create, Read, Update, and Delete (CRUD) operations inside the SQLite Database (browser.db).

   Occurrence: **BrowserDataContext, Bookmarks, History, HomePage,** and **Users** classes

Libraries Used to Create the Graphical User Interface (GUI):

1. **System.Windows.Forms** is a .NET framework library used to create, design and assign event handling to the graphical interface, including buttons, text boxes, labels, and list boxes.

   Occurrence: **Window.cs** and **Window.Designer.cs**

2. **Microsoft.VisualBasic** is used to create user input prompts for various features.
   Example: Editing Bookmarks, Adding User, Changing password
   Occurrence: **Bookmarks** and **Users** classes

CONCLUSION

Within a modular and persistent framework, the Simple Web Browser effectively illustrates fundamental web features including bookmark management, history tracking, URL navigation, and user-based homepage customization.

I am incredibly proud of the extent of error handling demonstrated in the code and the color customization of the status codes. The tooltips and context menus make the browser intuitive and user-friendly.

Given the opportunity to redo this project, I would work harder to resolve the Linq connection issues and make the database as required by the specification.

All things considered, the project succeeds in offering a lightweight, useful browser with a clear user interface and well-organized architecture, setting the stage for upcoming improvements like history navigation across sessions.

REFERENCES

External Dependencies

Libraries:

1. System.Data.SQLite
2. System.Data.SQLite.Linq
3. Microsoft.VisualBasic